Aufgabe 1: Drehfreudig

Team-ID: 25ITM8_TeamX

Team: Tomas Pham und Anil Djojan (Team7)

Bearbeiter/-innen dieser Aufgabe:
Tomas Pham

16. September 2025

Inhaltsverzeichnis

Lösungsidee 1
Umsetzung 1
Beispiele 2
Quellcode 2

Lösungsidee

Kurze Zusammenfassung Aufgabestellung:

Bei der Aufgabe 1: “Drehfreudig” wird ein Algorithmus erwartet, der überprüft, ob ein Baum drehfreudig ist. Dies bedeutet, dass exakt der selbe Baum um 180 Grad rotiert und nochmal dargestellt wird. Wenn sich die Blätter der beiden Bäumen exakt auf der selben Stelle berühren, dann erfüllt er das Kriterium der „Drehfreudigkeit“.

Erste Gedanken:

Da in der vorliegenden Aufgabenstellung „Bäume“ die zentralle Rolle darstellen, fallen mir folgende Ideen ein:

> „Binary Trees“ bzw. Binärerbaum als Datenstruktur, da diese im Aufbau und Strukur sehr ähnlich sind: Wurzel, Knoten und Blätter. Dabei stellt die Wurzel den Anfangspunkt dar, vom dem aus ein linker bzw. rechter Teilbaum ausgeht. Ist ein Teilbaum leer, wird dieser als fehlendes Kind bezeichnet. Wenn ein Knoten weder ein linkes noch ein rechtes Kind hat, dann ist dies das Blatt.
>
> Allerdings beschränkt sich diese Idee leider nur auf Binäre Bäume, also ein Baum der höchstens zwei Kindknoten hat. In der vorliegende Aufgabe „Drehfreudig“ gibt es Testfälle bei dem ein Knoten bzw. die Wurzel drei Kindknoten besitzt.

Dennoch führt diese Idee zu einem wichtigen Schlagwort: Rekursion. Darunter versteht man einen Vorgang, bei dem sich eine Methode so lange selbst aufruft bzw. ausgeführt wird, bis eine bestimmte Bedingung erfüllt ist.

Für die vorliegende Aufgabe ist Rekursion besonders geeignet, da ein Baum eine rekursive Struktur besitzt. Er besteht aus einer Wurzel und Blättern, und jeder Teilbaum folgt demselben Aufbau wie der gesamte Baum. Dadurch kann man eine Methode schreiben, die lediglich die Grundstruktur des Baumes darstellt und sich anschließend so lange selbst aufruft, bis der gesamte Baum durch die Teilbäume konstruiert ist.

Ein solcher Ansatz ist flexibel und nicht „hartcodiert“, also eine Methode, die sich genau nur auf einen exakten Fall anwenden lässt. Daher würde sich diese Methode somit auf beliebige Testfälle anwenden lassen und kann aus unterschiedlichen Eingabedaten jeweils den passenden Baum erzeugen. Die Rekursion bezieht sich hierbei nicht nur auf den Aufbau des Baums, sondern kann ebenso bei der Traversierung von Bäumen angewendet werden, um auf diese Weise beispielsweise die Drehfreudigkeit herauszufinden.

Nachdem eine Idee für die Konstruktion sowie Traversierung der Bäume herausgestellt wurde, stellt sich nun die Frage: „Wie lässt sich die Drehfreudigkeit eines Baums herausfinden ? “.

Lösungsidee:

Mögliche Idee wäre, die Drehfreudigkeit eines Baums durch die struktureller Symmetrie zu ermitteln. Ein Baum gilt als drehfreudig, wenn er nach einer Drehung um 180 Grad exakt dieselbe Form aufweist wie zuvor, also wenn seine Äste, Knoten und Blätter in spiegelbildlicher Anordnung zueinanderstehen.

Hierfür wäre die Idee eines Palindroms der Baum-Breite sein. Ein Palindrom ist eine Zeichen- oder Zahlenfolge, die vorwärts und rückwärts genau gleich gelesen wird. Überträgt man dieses Prinzip auf die Baumstruktur, bedeutet das:

Wenn man den Baum „spiegelt“ bzw. um 180 Grad dreht, sollten die Knoten in ihrer relativen Anordnung identisch bleiben, also wie bei den Buchstaben eines Palindroms, die sich symmetrisch um die Mitte spiegeln.

Um diese Eigenschaft zu überprüfen, wird der Baum in Teilbäume zerlegt. Jeder Teilbaum wird dabei als eigenständige Einheit betrachtet, die wiederum aus kleineren Teilstrukturen besteht. Diese rekursive Vorgehensweise ist entscheidend, da sich ein Baum immer wieder aus der selben Grundstruktur zusammensetzt: Wurzel, Knoten und deren Kinder.

Ist dieses Strukturmuster symmetrisch, also liest es sich von links nach rechts genauso wie von rechts nach links, dann spiegelt dieser Teilbaum seine Struktur. Wird diese Bedingung nicht nur an der Wurzel, sondern rekursiv in allen Ebenen des Baums erfüllt, bedeutet das, dass der gesamte Baum bei einer Drehung um 180° identisch zu sich selbst bleibt.

Damit lässt sich die Drehfreudigkeit eines Baums als rekursive Überprüfung der symmetrischen Breitenverteilung seiner Teilbäume verstehen. Allgemein lässt sich also

sagen, dass eine mehrstufige, strukturelle Palindromprüfung stattfindet, die die Drehfreudigkeit des gesamten Baums herausstellt.

Umsetzung

Die Umsetzung der zuvor beschriebenen Lösungsidee erfolgt in mehreren logisch aufeinander aufbauenden Schritten, die den beschriebenen rekursiven Ansatz und die Palindrom-Logik darstellen. Der Algorithmus ist in Javascript bzw. TypeScript umgesetzt und nutzt eine rekursive Baumstruktur, um die einzelnen Ebenen des Baums systematisch zu analysieren.

1. Einlesen und Aufbau des Baums

Zuerst wird die String Eingabe, die den Baum als Klammerstruktur darstellt (z. B. ((()())(()())), rekursiv in einen Javascript Objekt umgewandelt.

Dies wird durch die Funktion „*parseTree()*" ermöglicht, die jede öffnende Klammer „(„ als neuen Knoten interpretiert und zugehörige Teilbäume erkennt, bis die passende schließende Klammer „)" erreicht ist. Dadurch entsteht eine hierarchische Struktur aus Knoten und Kindknoten, die den gesamten Baum repräsentiert.

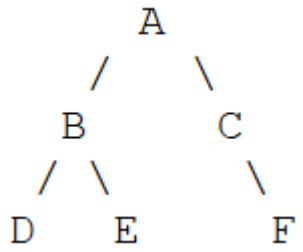
Dieses Vorgehen sorgt dafür, dass der Algorithmus mit beliebig tiefen oder unterschiedlich verzweigten Bäumen umgehen kann.

2. Zuweisung von Intervallen (Breiten) zu den Knoten des Baums

Anschließend erhält jeder Knoten des Baums durch die Funktion „*assignIntervals()*" zwei Werte – l (linke Grenze) und r (rechte Grenze). Diese Werte bilden ein Intervall, das angibt, welchen „Breitenbereich" der Knoten innerhalb des Gesamtbaums einnimmt. Die Intervalle werden rekursiv vergeben:

- Blätter (Knoten ohne Kinder) erhalten ein Intervall der Länge 1,
- Übergeordnete Knoten erhalten Intervalle, die alle ihre Teilbäume umschließen.

Beispiel:



Blätter zuordnen:

D: l=1, r=1 → [1,1]

E: l=2, r=2 → [2,2]

F: l=3, r=3 → [3,3]

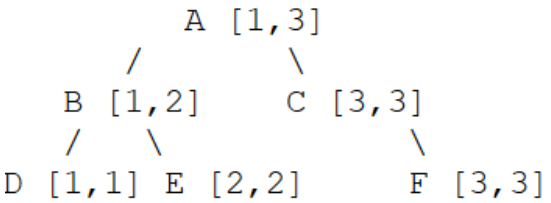
Elternknoten berechnen:

B → Kindknoten [D, E]: l=1, r=2 → Intervall [1,2]

C → Kindknoten [F]: l=3, r=3 → Intervall [3,3]

A → Kindknoten [B, C]: l=1, r=3 → Intervall [1,3]

Ergebnis mit den zugewiesenen Intervallen:



3. Ermittlung des Breitenmuster

Für jeden Knoten wird anschließend die Breite jedes Kindknotens mithilfe der Funktion „*calcWidth()*“ berechnet, also wie viel Raum dieser innerhalb des Baums einnimmt.

Dabei wird die Breite durch die Differenz zwischen rechter und linker Grenze berechnet und anschließend mit 1 addiert, damit die Knoten selbst einbezogen werden:

$$Breite = r - l + 1$$

Zum vorliegenden Beispiel, die berechneten Ergebnisse zu den einzelnen Knoten:

Knoten	l	r	calcWidth(node) = r-l+1
D	1	1	1
E	2	2	1
F	3	3	1
B	1	2	2
C	3	3	1
A	1	3	3

B → Kindknoten [D, E] → Breiten [1, 1] → Palindrom

C → Kindknoten [F] → Breiten [1] → Palindrom

A → Kindknoten [B, C] → Breiten [2, 1] → kein Palindrom

D, E und F werden nicht überprüft, da sie keine nachfolgende Kindknoten besitzen und daher stellen sie die „Blätter“ dar.

5. Gesamtablauf und Ergebnis

Der Algorithmus arbeitet vollständig rekursiv und prüft in jeder Ebene, ob: alle Unterstrukturen korrekt aufgebaut sind, die Breiten der Teilbäume eine symmetrische (palindromische) Anordnung besitzen.

Wenn alle Prüfungen erfolgreich sind, wird der Baum als drehfreudig erkannt. Im vorliegenden Beispiel lässt sich feststellen, dass die Knoten B und C zwar einen Palindrom erfüllen, jedoch der Knoten A nicht. Daher gilt der gesamte Baum als nicht drehfreudig.

Zudem protokolliert das Programm die einzelnen Schritte (z. B. Parsen, Intervallzuweisung, Prüfung) über ein Logging-System, um den Ablauf nachvollziehbar zu machen.

Werkzeuge

- *Javascript und Node.js*

Für diese Aufgabe wird lediglich ein Programm mit einem Algorithmus benötigt, das auf verschiedene Testfälle angewendet werden kann, um die Drehfreudigkeit eines Baums zu überprüfen. Aus meiner Sicht ist es sinnvoll, das Programm so minimalistisch, effizient und leicht wartbar wie möglich zu gestalten, da eine aufwendige GUI (Graphical User Interface) oder zusätzliche Frameworks für diese Aufgabe nicht notwendig sind.

Da ich bereits mit JavaScript vertraut bin, habe ich das Programm in dieser Sprache umgesetzt. JavaScript wird üblicherweise im Browser ausgeführt, wodurch theoretisch auch eine grafische Benutzeroberfläche möglich wäre. Dies würde jedoch deutlich mehr Ressourcen und Leistung erfordern, was für die Anforderungen dieser Aufgabe die Verhältnismäßigkeit übertrifft. Um die Effizienz zu erhöhen und Ressourcen zu sparen, habe ich mich daher für Node.js entschieden.

Node.js ist eine serverseitige Laufzeitumgebung für JavaScript, die unabhängig vom Browser arbeitet und die Ausführung über die Kommandozeile (CLI = Command Line Interface) ermöglicht. Eine CLI-Anwendung ist für diesen Zweck besonders geeignet, da sie einfach bedienbar, schnell und ressourcenschonend ist. Frameworks wie React, Angular oder Vue wurden daher bewusst ausgeschlossen, da sie für eine reine Konsolenanwendung unnötig komplex und ressourcenintensiv wären.

- *Typescript*

Ein weiterer wichtiger Punkt war die Entscheidung für TypeScript anstelle von reinem JavaScript. JavaScript ist dynamisch typisiert, wodurch leicht Fehler auftreten können, wie beispielsweise durch falsche Datentypen, vergessene Argumente oder fehlerhafte Funktionsaufrufe. TypeScript bietet in diesem Zusammenhang mehrere entscheidende Vorteile: Es sorgt für Typensicherheit, sodass falsche Datentypen bereits während der Entwicklung erkannt werden. Dies reduziert die Fehleranfälligkeit erheblich. Des Weiteren liefert TypeScript detailiertere Fehlermeldungen, was das Debugging bzw. Fehlerbehebung erleichtert und den Entwicklungsprozess deutlich beschleunigt.

Durch die statische Typenüberprüfung wird der Code zudem besser strukturiert, lesbarer und wartbarer, insbesondere bei späteren Anpassungen oder Erweiterungen. TypeScript selbst ist keine eigenständige Programmiersprache, sondern wird zur Compile-Zeit in reguläres JavaScript übersetzt, das anschließend in Node.js ausgeführt wird. Zusätzlich bietet TypeScript durch Features wie Autovervollständigung weitere Vorteile bei der Entwicklung, insbesondere bei größeren Projekten.

- *ESLint*

Zur Sicherstellung von einheitlicher Codequalität und Lesbarkeit kommt es zum Einsatz von ESLint. Dieser prüft den Code auf Stil- und Strukturregeln und hilft dabei, Best Practices einzuhalten, sodass der Code konsistent, verständlich und leichter wartbar bleibt.

Für die Darstellung der Konsolenausgabe, wurden noch folgende Dependencies bzw. Abhängigkeiten verwendet:

- *chalk*: Ermöglicht das die farbliche Darstellung von Texten in der Konsole, wodurch wichtige Informationen, Statusmeldungen oder Fehler hervorgehoben werden können.
- *gradient-string*: Diese Abhängigkeit ermöglicht farbige Textverläufe in der Konsole und sorgt so für individuellere und ansprechendere Ausgaben
- *figlet*: Erstellt ASCII-Art-Schriften in der Konsole, beispielsweise um den Programmtitel beim Start auffällig darzustellen und die CLI ansprechender zu gestalten. In meinen Projekt wurde der Titel „Tomas Pham A1-Drehfreudigkeit-BWINF 25“ beispielweise durch auflälliger durch die Größen und Schriftart dargestellt.
- *prompt*: Ermöglicht interaktive Benutzereingaben in der Konsole. So kann der Nutzer beispielsweise die Test-Dateien direkt in der Konsole mithilfe der Steuerung der Tastatur auswählen und diese dann durch das Bestätigen ausführen.
- *rimraf*: Stellt eine plattformunabhängige Möglichkeit bereit, Dateien oder Verzeichnisse rekursiv zu löschen, ähnlich wie der Befehl rm -rf. Dies ist nützlich für temporäre Dateien

oder zum Zurücksetzen von Testumgebungen. In meinem Fall wurde es für den dist-Ordner verwendet, der vor jedem neuen Build gelöscht und anschließend neu erstellt wird. Dadurch wird verhindert, dass Fehler durch veraltete Dateien, Caches oder Build-Artefakte entstehen.

Insgesamt war das Ziel der Umsetzung, einfaches, effizientes und gut wartbares CLI-Programm zu erstellen, das mit minimalem Overhead arbeitet. Dabei wird die Drehfreudigkeit von Bäumen zuverlässig mithilfhe einer klaren Struktur, Typensicherheit und ein nachvollziehbares Logging geprüft werden.

- *Unterstützung durch KI-gestützte Werkzeuge*

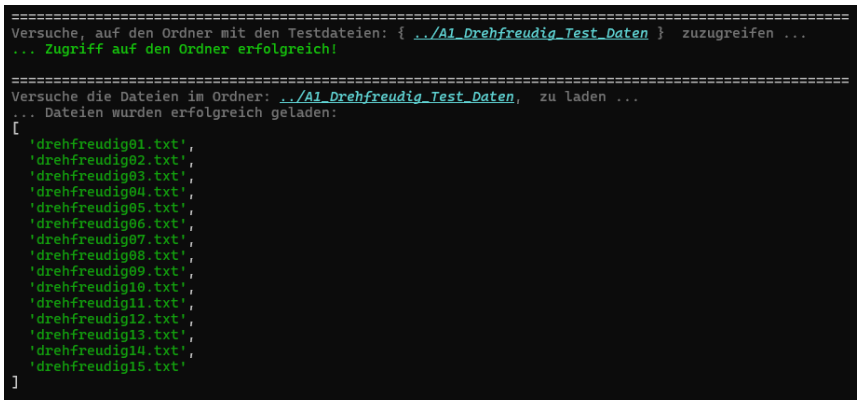
Die grundlegende Idee des Algorithmus wurde auf Basis eines konzeptionellen Vorschlags von ChatGPT 4o entwickelt und anschließend in der Entwicklungsumgebung Visual Studio Code mit Unterstützung von GitHub Copilot v1.388.0 umgesetzt. GitHub Copilot wurde hierbei insbesondere zur automatischen Codevervollständigung und zur Unterstützung bei wiederkehrenden Routineaufgaben verwendet. Dadurch konnte der manuelle Aufwand für standardisierte Codetätigkeiten reduziert werden, sodass der Fokus stärker auf die Ausarbeitung, der strukturellen Gestaltung und der algorithmischen Logik des Programms gesetzt wurde.

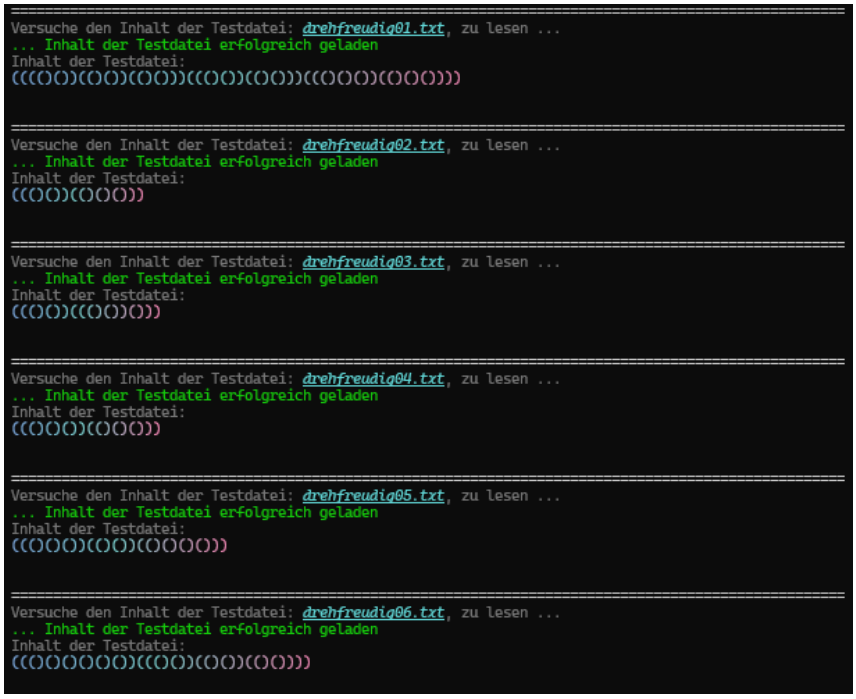
Beispiele

Im Rahmen der Aufgabe wurden die offiziellen Beispieldateien des Bundeswettbewerbs Informatik (BWINF) aus dem Ordner „*../A1_Drehfreudig_Test_Daten“* eingelesen und analysiert.

Das Programm durchsucht automatisch den angegebenen Ordner, lädt alle vorhandenen Testdateien und listet sie übersichtlich in der Konsole auf. Anschließend kann der Benutzer auswählen, welcher Testfall ausgeführt werden soll.

Beim Start zeigt das Programm zunächst die erfolgreiche Initialisierung und das Einlesen der Dateien:

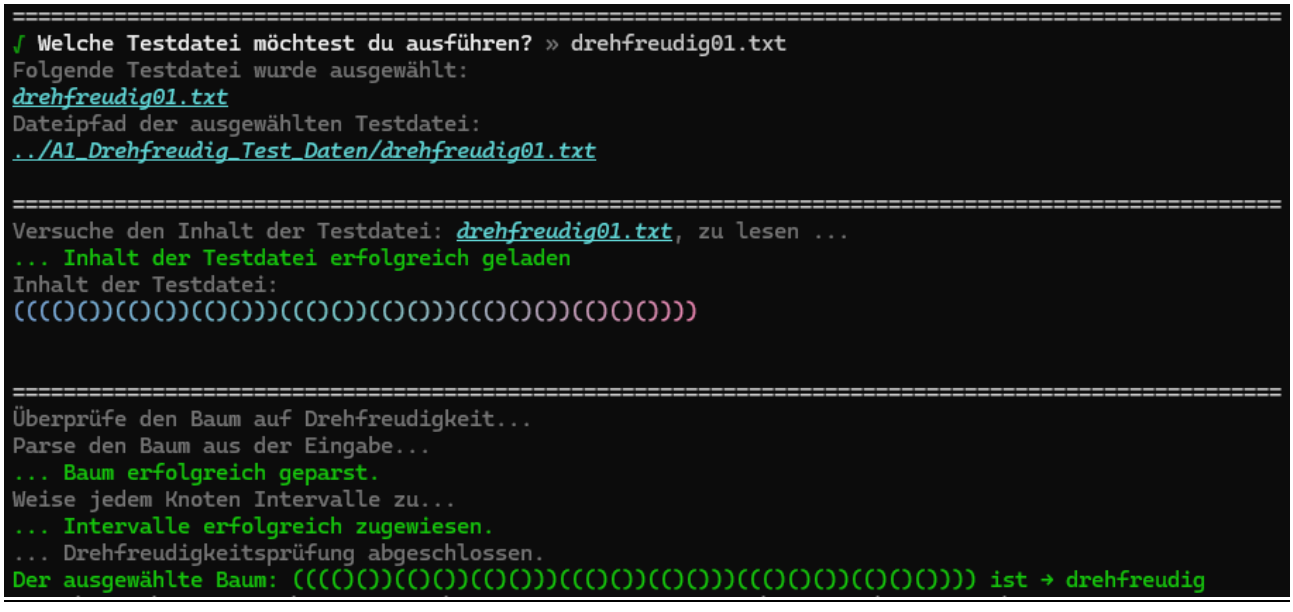






Damit wird sichergestellt, dass alle bereitgestellten Eingabedateien verfügbar und lesbar sind. Anschließend kann der Nutzer mithilfe der Tastatursteuerung, mit den Pfeiltasten nach oben und nach unten durch die Dateien navigieren. Mit Enter wird dann die Drehfreudigkeit-Überprüfung für die ausgewählte Datei gestartet, und ein entsprechendes Ergebnis geliefert:

Drehfreudiges Beispiel:



Nicht drehfreudiges Beispiel:

```
================================================================================================
✓ Welche Testdatei möchtest du ausführen? » drehfreudig02.txt
Folgende Testdatei wurde ausgewählt:
drehfreudig02.txt
Dateipfad der ausgewählten Testdatei:
../A1_Drehfreudig_Test_Daten/drehfreudig02.txt

================================================================================================
Versuche den Inhalt der Testdatei: drehfreudig02.txt, zu lesen ...
... Inhalt der Testdatei erfolgreich geladen
Inhalt der Testdatei:
(((()))(()()()))


================================================================================================
Überprüfe den Baum auf Drehfreudigkeit...
Parse den Baum aus der Eingabe...
... Baum erfolgreich geparst.
Weise jedem Knoten Intervalle zu...
... Intervalle erfolgreich zugewiesen.
... Drehfreudigkeitsprüfung abgeschlossen.
Der ausgewählte Baum: (((()))(()()())) ist → nicht drehfreudig
```

Ergebnis der Testdateien aus der BWINF Aufgabe 44:

'drehfreudig01.txt', → drehfreudig

'drehfreudig02.txt', → nicht drehfreudig

'drehfreudig03.txt', → nicht drehfreudig

'drehfreudig04.txt', → drehfreudig

'drehfreudig05.txt', → nicht drehfreudig

'drehfreudig06.txt', → drehfreudig

'drehfreudig07.txt', → nicht drehfreudig

'drehfreudig08.txt', → drehfreudig

'drehfreudig09.txt', → nicht drehfreudig

'drehfreudig10.txt', → drehfreudig

'drehfreudig11.txt', → nicht drehfreudig

'drehfreudig12.txt', → nicht drehfreudig

'drehfreudig13.txt', → drehfreudig

'drehfreudig14.txt', → nicht drehfreudig

'drehfreudig15.txt' → drehfreudig

Die Tests zeigen, dass der Algorithmus in allen Fällen korrekt arbeitet. Dabei werden einfache als auch komplexe Baumstrukturen zuverlässig auf Drehfreudigkeit überprüft. Die rekursive

Intervallzuweisung in Kombination mit der Breitenmusteranalyse liefert konsistente Ergebnisse, die exakt den Vorgaben der BWINF-Beispiele entsprechen.

Quellcode

fileManager.ts

Diese Datei übernimmt die komplette Verwaltung der Testdateien aus dem Ordner ../A1_Drehfreudig_Test_Daten.

Sie stellt sicher, dass alle Dateien geladen, eingelesen und dem Benutzer zur Auswahl angezeigt werden können.

```
import { promises as fs } from "fs";
import chalk from "chalk";
import prompts from "prompts";
import { logger } from "./logger.js";
import { teen } from "gradient-string";
```

Diese Zeilen importieren sowohl Standard-Module wie „fs“ von Node.js als auch externe npm-Pakete wie z.b. „z. B. chalk, prompts, gradient-string“ und lokale Projektmodule wie „logger“s. Dadurch werden Funktionen und Objekte aus anderen Modulen in das aktuelle Modul eingebunden, um sie dort weiterzuverwenden.

```
async function loadTestFiles(
  folderPath: string
): Promise<string[] | undefined> {
  logger.separator();
  try {
    logger.info(
      `Versuche die Dateien im Ordner: ${chalk.italic.underline.bold.cyan(
        folderPath
      )},  zu laden ...`
    );
    const files = (await fs.readdir(folderPath)).filter(
      (f) => f !== ".DS_Store"
    );
    if (!files || files.length === 0) {
      logger.error("... Keine Dateien im Ordner gefunden.");
      logger.error(
        `Bitte füge Testdateien in den folgenden Ordner: ${chalk.italic.underline.bold.cyan(
          folderPath
        )}`
      );
      return;
    }
    logger.info("... Dateien wurden erfolgreich geladen:", files);
    return files;
  } catch (error) {
    logger.error("... Fehler beim Laden der Dateien:", error);
    return undefined;
  }
}
```

Die Funktion „*loadTestFiles(folderPath)*" lädt alle Testdateien aus einem angegebenen Ordner (folderPath).

- fs.readdir() liest den Inhalt des Ordners.
- Mit .filter() werden unnötige Systemdateien (wie .DS_Store auf macOS) herausgefiltert.
- Jede geladene Datei wird durch den Logger in der Konsolenausgabe dargestellt.

```
    if (!files || files.length === 0) {
      logger.error("... Keine Dateien im Ordner gefunden.");
      logger.error(
        `Bitte füge Testdateien in den folgenden Ordner: ${chalk.italic.underline.bold.cyan(
          folderPath
        )}`
      );
      return;
    }
```

Falls der Ordner leer ist oder ein Fehler auftritt, gibt die Funktion eine Fehlermeldung aus und fordert den Nutzer, die entsprechende Testdateien in den Ordner hinzuzufügen.

```
    logger.info("... Dateien wurden erfolgreich geladen:", files);
    return files;
```

Wenn alles problemlos durchgelaufen ist, werden die gefundenen Dateinamen als Array zurückgegeben.

```
async function readTestFiles(filePath: string) {
  logger.separator();
  try {
    logger.info(
      `Versuche den Inhalt der Testdatei: ${chalk.italic.underline.bold.cyan(
        filePath
      )}, zu lesen ...`
    );
    const fileContent = await fs.readFile(
      `../A1_Drehfreudig_Test_Daten/${filePath}`,
      "utf-8"
    );
    logger.success("... Inhalt der Testdatei erfolgreich geladen");
    logger.info("Inhalt der Testdatei:", teen(fileContent));
    return fileContent.trim();
  } catch (error) {
    logger.error("... Fehler beim Lesen der Testdatei:", error);
  }
}
```

Die Funktion *readTestFiles(filePath)* liest den Textinhalt einer einzelnen Testdatei aus.

- Mit fs.readFile() wird die Datei asynchron geladen.
- Der Dateipfad wird dynamisch zusammengesetzt, sodass alle Dateien aus dem Testordner stammen.
- Das Ergebnis wird als UTF-8-String zurückgegeben.
- Der Inhalt wird mit einem farbigen Verlauf (gradient-string) in der Konsole angezeigt.
- .trim() entfernt Leerzeichen oder Zeilenumbrüche am Anfang und Ende des Inhalts.

Fehler beim Zugriff werden abgefangen und klar im Log angezeigt.

fileManager()

```
export async function fileManager() {
  const testFolderPath = "../A1_Drehfreudig_Test_Daten";
  logger.separator();
  try {
    logger.info(
      `Versuche, auf den Ordner mit den Testdateien: { ${chalk.italic.underline.bold.cyan(
        testFolderPath
      )} }  zuzugreifen ...`
    );
    await fs.access(testFolderPath);
    logger.success("... Zugriff auf den Ordner erfolgreich!");
  } catch (error) {
    logger.error("... Zugriff auf den Ordner fehlgeschlagen:", error);
  }
```

Zuerst wird geprüft, ob der Testdaten-Ordner existiert und ob das Programm Lesezugriff auf den Ordner hat.

- fs.access() überprüft, ob der Ordner vorhanden und zugänglich ist.

```
const testFiles = await loadTestFiles(testFolderPath);

if (testFiles) {
  for (const testFile of testFiles) {
    await readTestFiles(testFile);
  }
```

Dann wird die oben beschriebene Funktion *loadTestFiles()* aufgerufen, um alle Dateinamen in einem Array zu erhalten welche unter der Variable *testFiles* gespeichert wird. Dabei wird dann jede Datei iterrativ einmal kurz eingelesen und angezeigt, um dem Nutzer den Überblick zu geben, welche Inhalte verfügbar sind.

```
const userResponse = await prompts({
  type: "select",
  name: "testFile",
  message: "Welche Testdatei möchtest du ausführen?",
  choices: testFiles.map((testFile) => ({
    title: testFile,
    value: testFile,
  })),
  initial: 0,
});
logger.info(
  "Folgende Testdatei wurde ausgewählt:",
  chalk.italic.underline.bold.cyan(userResponse.testFile)
);

if (!userResponse.testFile) {
  logger.error(
    chalk.red("Keine Testdatei ausgewählt. Vorgang abgebrochen.")
  );
  return;
}
```

Mit der prompts-Bibliothek wird ein interaktives Auswahlmenü geöffnet, wodurch der Benutzer bequem eine der geladenen Testdateien in der Konsole auswählen kann .

- choices enthält alle Dateinamen.
- initial: 0 bedeutet, dass die erste Datei standardmäßig vorausgewählt ist.

Der ausgewählte Dateiname wird in farbigem Text ausgegeben. Im Falle, dass der Benutzer keine Auswahl trifft, wird der Vorgang abgebrochen.

```
  const selectedFilePath = `${testFolderPath}/${userResponse.testFile}`;
  logger.info(
    "Dateipfad der ausgewählten Testdatei:",
    chalk.italic.underline.bold.cyan(selectedFilePath)
  );

  const fileContent = await readTestFiles(userResponse.testFile);
  if (!fileContent) {
    logger.error("Kein Inhalt in der Datei gefunden.");
    return;
  }
  logger.separator();
  return fileContent;
}
```

Schließlich wird der Inhalt (als String) zur weiteren Verarbeitung an das Hauptprogramm in der *index.ts* Datei zurückgegeben, wo später die Drehfreudigkeitsprüfung erfolgt.

treeParser.ts

```ts
type Node = {
  children: Node[];
  l?: number;
  r?: number;
  parent?: Node | null;
};
```

Zunächst wird ein eigener Typ Node definiert, der den Aufbau eines einzelnen Baumknotens beschreibt.
Jeder Knoten enthält:

- children: Node[] → ein Array seiner Kindknoten,
- l?: number und r?: number → optionale linke und rechte Intervallgrenzen
- parent?: Node | null → ein Verweis auf den Elternknoten (für die Baumstruktur).

```ts
function parseTree(treeString: string, currentPos = 0): [Node, number] {
  if (treeString[currentPos] !== "(")
    throw new Error(`Erwarte '(' an Position ${currentPos}`);

  const node: Node = { children: [], parent: null };
  currentPos++;

  while (treeString[currentPos] === "(") {
    const [child, newPos] = parseTree(treeString, currentPos);
    child.parent = node;
    node.children.push(child);
    currentPos = newPos;
  }
  if (treeString[currentPos] !== ")")
    throw new Error(`Erwarte ')' an Position ${currentPos}`);
  currentPos++;
  return [node, currentPos];
}
```

Dies Funktion *parseTree(treeString, currentPos)* übersetzt die Klammernotation in eine Baumstruktur aus Node-Objekten.

- Parameter:
  - treeString: der Eingabestring (z. B. "((()())(()))").
  - currentPos: der aktuelle Index im String, an dem gerade gelesen wird.
- Rückgabewert:

- Ein Tupel [Node, number], bestehend aus:
  - dem geparsten Node-Objekt (Wurzel des aktuellen Teilbaums),
  - und der Position im String, an der das Parsen endet.

Die Funktionsweise im Detail:

Zunächst wird überprüft, ob an der aktuellen Position tatsächlich eine öffnende Klammer '(' steht. Wenn nicht, wird ein Fehler ausgegeben, welcher fehlerhafte Eingaben verhindert.

Danach wird ein neuer Knoten node erzeugt, welcher zunächst keine Kinder (children: []) sowie keinen Elternverweis (parent: null) besitzt.

Die Position currentPos wird um eins erhöht, um das '(' zu überspringen und auf nächstes Zeichen zu überzugehen.

Anschließend wird in einer Schleife geprüft, ob an der neuen Position erneut '(' steht.

- Falls ja, bedeutet das: Der aktuelle Knoten hat ein Kind.
- Für jedes Kind wird rekursiv wieder parseTree aufgerufen.
- Nach jedem rekursiven Aufruf wird der Kindknoten (child) mit child.parent = node dem aktuellen Knoten zugewiesen und zur children-Liste hinzugefügt.

Sobald keine öffnende Klammer '(' mehr folgt, wird eine schließende Klammer ')' erwartet.
Falls nicht dies nicht der Fall sein sollte, wird erneut ein Fehler asugegeben („Erwarte ')"“).

Am Ende wird die Position um eins erhöht aufgrund der schließende Klammer ')' und [node, currentPos] zurückgegeben.

Damit wird der gesamte Klammerausdruck systematisch in einen hierarchischen Baum aus verschachtelten Objekten umgewandelt.

```
function assignIntervals(node: Node, start = 1): number {
  if (node.children.length === 0) {
    node.l = start;
    node.r = start;
    return start + 1;
  }
  let currentCount = start;
  for (const child of node.children) {
    currentCount = assignIntervals(child, currentCount);
  }
  node.l = start;
  node.r = currentCount - 1;
  return currentCount;
}
```

Die Funktion *assignIntervals(node, start)* weist jedem Knoten ein numerisches Intervall [l, r] zu, das seine Position und Breite innerhalb des gesamten Baums beschreibt.

- Parameter:
    - node: der aktuelle Knoten.
    - start: die Startposition des Intervalls (Standardwert 1).
- Rückgabewert:
    - Eine Zahl, die die nächste freie Startposition zurückgibt – also r + 1.

Wenn der aktuelle Knoten keine Kinder besitzt, also ein Blatt ist, erhält er ein Intervall der Länge 1, das von start bis start reicht.
Jedoch im Falle, dass der Knoten Kinder besitzt, wird zunächst currentCount auf den Startwert gesetzt.

Anschließend wird für jeden Kindknoten *assignIntervals()* rekursiv aufgerufen.
Der Rückgabewert dieser Aufrufe gibt die neue rechte Grenze des zuletzt verarbeiteten Kindes an und wird als neuer currentCount übernommen.
Nach der Verarbeitung aller Kinder wird node.l = start und node.r = currentCount - 1 gesetzt, da der Zähler im letzten Kindaufruf bereits um 1 erhöht wurde.
Schließlich gibt die Funktion currentCount zurück, also die nächste verfügbare Position nach dem Ende des aktuellen Intervalls. Diese Intervalle bilden die Grundlage für die spätere Berechnung der Breite jedes Teilbaums.

```
function calcWidth(node: Node): number {
  if (node.l === undefined || node.r === undefined)
    throw new Error("Intervall nicht gesetzt");
  return node.r - node.l + 1;
}
```

Die Funktion *calcWidth(node: Node)* dient zur Berechnung der Breite eines Knotens auf Grundlage seiner zuvor zugewiesenen Intervallgrenzen.

Als Parameter erhält sie den Knoten, dessen Breite ermittelt werden soll. Dabei überprüft die Funktion zunächst, ob die Intervallgrenzen l (linke Grenze) und r (rechte Grenze) bereits gesetzt sind. Ist dies nicht der Fall, wird ein Fehler ausgelöst, um sicherzustellen, dass die Funktion assignIntervals() zuvor aufgerufen wurde und die notwendigen Werte korrekt initialisiert wurden.

Wenn beide Grenzen vorhanden sind, berechnet die Funktion die Breite des Knotens durch die Formel node.r - node.l + 1. Dieses Ergebnis entspricht der Länge des Intervalls und gibt an, wie viele Positionen der Teilbaum des Knotens in der Gesamtstruktur einnimmt.

```
function checkPalindrome(intervalWidths: number[]): boolean {
  let i = 0;
  let j = intervalWidths.length - 1;
  while (i < j) {
    if (intervalWidths[i] !== intervalWidths[j]) return false;
    i++;
    j--;
  }
  return true;
}
```

Die Funktion *checkPalindrome(intervalWidhths)* überprüft, ob eine gegebenes Array von Breitenwerten symmetrisch ist, also ein Palindrom bildet. Als Parameter erhält sie ein Array intervalWidths, das die Breiten der Kindknoten eines Knotens enthält. Der Rückgabewert ist ein boolescher Wert: Die Funktion gibt true zurück, wenn das Array spiegelbildlich aufgebaut ist, ansonsten false.

Der Ablauf basiert auf einem Zwei-Zeiger-Prinzip: Ein Zeiger i startet am linken Ende im Array, während ein zweiter Zeiger j am rechten Ende beginnt. Beide Zeiger bewegen sich schrittweise aufeinander zu. In jeder Iteration werden die Elemente intervalWidths[i] und intervalWidths[j] miteinander verglichen. Sobald ein Unterschied festgestellt wird, wird die Funktion sofort mit dem Ergebnis false beendet.

Wenn die Schleife hingegen ohne Abbruch durchläuft, bedeutet das, dass alle gegenüberliegenden Elemente übereinstimmen und somit ist das Array mit den Intervallen symmetrisch.

Diese Funktion bildet den zentralen Bestandteil der Drehfreudigkeitsprüfung, da sie erkennt, ob die Breitenverteilung der Kindstrukturen eines Knotens spiegelbildlich und potenziell drehfreudig ist.

```
export function checkDrehfreudig(treeString: string): {
  resultMessage: string;
  isDrehfreudig: boolean;
} {
  logger.info("Überprüfe den Baum auf Drehfreudigkeit...");
  logger.info("Parse den Baum aus der Eingabe...");
  const [root, pos] = parseTree(treeString, 0);
  if (pos !== treeString.length) {
    throw new Error("Ungültige Eingabe: Überzählige Zeichen am Ende");
  }

  logger.success("... Baum erfolgreich geparst.");
  logger.info("Weise jedem Knoten Intervalle zu...");

  assignIntervals(root, 1);
  logger.success("... Intervalle erfolgreich zugewiesen.");

  function checkNode(currentNode: Node): boolean {
    if (currentNode.children.length === 0) return true;
    const childWidths = currentNode.children.map(calcWidth);
    if (!checkPalindrome(childWidths)) return false;
    for (const child of currentNode.children) {
      if (!checkNode(child)) return false;
    }
    return true;
  }

  logger.info("... Drehfreudigkeitsprüfung abgeschlossen.");

  const isDrehfreudig = checkNode(root);
  let resultMessage = `Der ausgewählte Baum: ${treeString} ist `;

  if (isDrehfreudig) {
    resultMessage += "→ drehfreudig";
  } else {
    resultMessage += "→ nicht drehfreudig";
  }

  return { resultMessage, isDrehfreudig };
}
```

Die Funktion *checkDrehfreudig(treeString: string)* bildet den zentralen Bestandteil des Moduls und kombiniert alle zuvor definierten Hilfsfunktionen, um letztlich zu bestimmen, ob ein gegebener Baum drehfreudig ist oder nicht.

Als Eingabe erhält sie den Klammerausdruck des Baumes in Form eines Strings und als Rückgabewert liefert sie ein Objekt, das sowohl eine textliche Ergebnisnachricht (resultMessage) als auch das boolesche Prüfergebnis (isDrehfreudig) enthält.

Zu Beginn wird über den Logger ausgegeben, dass die Drehfreudigkeitsprüfung gestartet wurde. Anschließend wird der übergebene Klammerausdruck mithilfe der Funktion *parseTree* in eine hierarchische Struktur aus Node-Objekten umgewandelt. Sollte der Eingabestring nach dem Parsen noch nicht vollständig verarbeitet sein, wird ein Fehler geworfen, da dies auf eine ungültige Baumdarstellung hinweist.

Im nächsten Schritt werden durch den Aufruf von *assignIntervals(root, 1)* allen Knoten Intervalle zugewiesen. Diese Intervalle definieren die „linke“ und „rechte“ Grenze jedes Knotens innerhalb der Baumstruktur und bilden später die Grundlage für die Berechnung der Breiten der Teilbäume.

Daraufhin wird eine rekursive Hilfsfunktion *checkNode(currentNode)* definiert, welche die eigentliche Drehfreudigkeitsprüfung durchführt. Ist der aktuelle Knoten ein Blatt, gilt er automatisch als drehfreudig.

Für Knoten mit Kindern werden zunächst die Breiten der Kindknoten mithilfe von *calcWidth* ermittelt. Anschließend wird mit der Funktion *checkPalindrome(childWidths)* geprüft, ob die ermittelten Breiten symmetrisch angeordnet sind, also ob die Struktur der Kinder ein Palindrom bildet.

Sind die Breiten symmetrisch, wird *checkNode* rekursiv auf alle Kindknoten angewendet, um sicherzustellen, dass auch deren Teilbäume drehfreudig sind. Erst wenn alle Knoten diese Bedingung erfüllen, gilt der gesamte Baum als drehfreudig.

Nach Abschluss der Baumtraversierung enthält die Variable isDrehfreudig das Gesamtergebnis. Basierend darauf wird eine aussagekräftige Textnachricht zusammengesetzt: Ist der Baum drehfreudig, lautet die Ausgabe „→ drehfreudig“, sonst „→ nicht drehfreudig“.

Schließlich gibt die Funktion ein Ergebnisobjekt zurück, das sowohl die Nachricht als auch das boolesche Prüfergebnis enthält.

```
import { fileManager } from "./fileManager.js";
import { checkDrehfreudig } from "./treeParser.js";
import { logger } from "./logger.js";
import { setTimeout } from "node:timers/promises";

logger.separator();
await logger.title("TOMAS PHAM");
await setTimeout(1000);
await logger.title("A1 - Drehfreudig BWInf 25");
await setTimeout(1000);

const fileContent = await fileManager();
if (!fileContent) {
  logger.info(
    "Nachdem die Testdateien hochgeladen wurden, starte das Programm erneut."
  );
  process.exit(1);
}

const result = checkDrehfreudig(fileContent);
(result.isDrehfreudig ? logger.success : logger.error)(result.resultMessage);
```

Die Datei *index.ts* stellt das Hauptprogramm der Anwendung dar und steuert den gesamten Ablauf der Drehfreudigkeitsanalyse. also von der Initialisierung über das Laden der Testdateien bis hin zur abschließenden Überprüfung sowie Ergebnisdarstellung in der Konsole.

Zu Beginn werden mehrere Module importiert, die jeweils eine klar definierte Aufgabe übernehmen, wie beispielsweise die aus der eigenen Bibliothek *fileManager* oder *treeParser*.

Nach dem Programmstart gibt der Logger mit logger.separator() zunächst eine Trennlinie aus, um die Konsolenausgabe optisch zu strukturieren. Anschließend werden mit logger.title() zwei Titel angezeigt: Zuerst erscheint der Name des Autors („TOMAS PHAM“), gefolgt vom Aufgabentitel („A1 – Drehfreudig BWInf 25“). Zwischen den beiden Titelausgaben wird mithilfe von await setTimeout(1000) eine kurze Pause eingefügt, um die Lesbarkeit und Präsentation der Startsequenz zu verbessern.

Im nächsten Schritt ruft das Programm die Funktion *fileManager()* auf, um die Testdateien aus dem vorgesehenen Ordner zu laden. Der Benutzer kann dabei eine bestimmte Datei auswählen, deren Inhalt anschließend als String in der Variablen fileContent gespeichert wird. Sollte keine Datei ausgewählt oder der Zugriff fehlschlagen, informiert das Programm den Benutzer über den Logger, dass die Testdateien bereitgestellt werden müssen. Dabei wird die Ausführung mit process.exit(1) beendet.

Wenn eine Datei erfolgreich eingelesen wurde, wird ihr Inhalt an die Funktion *checkDrehfreudig(fileContent)* übergeben. Diese Funktion analysiert den übergebenen Baum und prüft rekursiv, ob er drehfreudig ist.

Das Ergebnis wird in Form eines Objekts zurückgegeben, das sowohl eine textliche Zusammenfassung (resultMessage) als auch ein boolesches Prüfergebnis (isDrehfreudig) enthält.

Zum Abschluss wertet das Hauptprogramm dieses Ergebnis aus und gibt die entsprechende Meldung in der Konsole aus. Ist der Baum drehfreudig, wird die Erfolgsnachricht mithilfe von logger.success() in grün dargestellt. Im Falle, dass der Baum nicht drehfreudig ist, wird über logger.error() eine rote Fehlermeldung ausgegeben.

Damit bildet die Datei *index.ts* die zentrale Steuereinheit des Projekts. Sie verbindet alle Hilfsfunktionen, steuert die Ausführungsreihenfolge, übernimmt die Benutzerinteraktion beim Datei-Import und sorgt für eine klare, farbig strukturierte Ergebnisdarstellung.